



*Akademia Górniczo-Hutnicza
w Krakowie*

Faculty: AGH	Academic Year 2025/2026	Year of study	Fields of science:
Student(s) name: Simon GANIER-LOMBARD & Daniil STEPANOV			
Course: Real Time Operating Systems in Control Applications			
Exercise nr: RTOS-04			
Exercise subject: Timers and interrupt handlers			
Lecturer: dr inż. Grzegorz Hayduk			
Date: 07.05.2026			Grade:

1 Exercise purpose

The goal of this exercise is we learn how to use timers in QNX.

We make a small program with one shot timer, then a cyclic timer, and last we add a second timer with a longer period to test it.

This helps us to understand how time-based tasks work in a real time system like in QNX.

2 Assumptions / Theory

A timer in QNX sends a signal or a pulse when the time is up.

To use a timer we need three steps:

- timer_create makes a new timer and gives back its id at the creation.
- timer_settime sets the first time and the repeat time.
- timer_delete removes the timer when we do not need it

If it_interval is zero the timer fires only one time.

And if it_interval is not zero the timer is cyclic and fires every period. One of smallest step is the system tick which is far less than 1 ms on fast CPUs.

3 Description of implementation

We made three small programs to do the three steps one by one.

- ex4_1.c is a one shot timer. It fires after 1.5 s, prints one message and the program ends.
 - ex4_2.c is the same program but cyclic. We set it_interval to 1.5 s so the timer fires every 1.5 s and we count the pulses.
 - ex4_3.c uses two timers on the same channel. Timer1 fires every 10 ms and timer2 fires every 10 s. Each timer has its own pulse code so we know which one fired. We count timer1 pulses and we expect exactly 999 of them inside one timer2 period.
- See the full commented code in the .zip file.

4 Conclusions

This exercise helped us understand how timers work in QNX. We learned the three steps.

We also learned the difference between one shot and cyclic timers, and how to use `itimerspec` to set the times.

Using two timers at the same time can be a bit harder because we had to check that the count of the fast timer was right inside the slow timer period. The pulse method was useful because the program is blocked and only wakes up when the timer fires, so the CPU is free for other tasks. The main drawback is that the timer step depends on the system tick.